

R4-Cyber-11 - Sécurisation de services réseaux

SlowLoris

Etudiant

GRONDIN Benjamin

Promotion

BUT 2 TP 1 Réseaux et télécommunications

Professeur

REPUSSEAU Willy

Table des matières

Introduction.....	2
Plan d'adressage.....	3
1.Installation d'Apache.....	3
1.1 - Choix du serveur.....	3
1.2 - Installation de apache.....	3
2.Installation de perl et slowLoris.....	4
2.1 - Installation de perl.....	4
2.2 - Installation de SlowLoris.....	4
3.Phase d'attaque.....	5
4. Analyse du trafic réseau avec WireShark.....	5
4.1 - Utilisation de wireshark.....	5
5. Installation de ModSecurity.....	7
5.1 - Installation de ModSecurity.....	7
Conclusion.....	9

Introduction

L'attaque Slowloris est une technique de déni de service (DoS) qui exploite les connexions persistantes pour saturer un serveur web avec un minimum de ressources. Ce TP a pour objectif de comprendre son fonctionnement, d'analyser son impact sur un serveur Apache et d'explorer des stratégies de détection et de mitigation. À travers une expérimentation pratique, nous verrons comment identifier et contrer cette menace efficacement.

Plan d'adressage

Machines	Adresses IP
Serveur web	192.168.50.127/24
Machine attaquante	192.168.50.197/24

1.Installation d'Apache

1.1 - Choix du serveur

Pour débiter ce TP sur HAProxy, il faut d'abord une distribution Linux pour héberger les sites web. J'ai donc choisi **Ubuntu**, car je suis plus à l'aise sur cette distribution de linux.

1.2 - Installation de apache

Pour tester l'attaque par *déni de service (DoS)*, nous commençons par installer un serveur *Apache*, qui servira de *machine cible*. Pour cela, nous exécutons les commandes suivantes :

```
#sudo apt update  
#sudo apt upgrade  
#sudo apt install apache2
```

Avant d'installer Apache, il ne faut pas oublier de mettre à jour les dépôts avec *update* et *upgrade*.

2.Installation de perl et slowLoris

Slowloris est une attaque DoS qui maintient des connexions HTTP incomplètes pour saturer un serveur web, et *Perl* est utilisé pour son efficacité dans la gestion des sockets et l'automatisation des requêtes réseau avec un minimum de ressources.

2.1 - Installation de perl

Pour commencer, nous installons Perl avec la commande suivante :

```
#sudo apt install perl
```

2.2 - Installation de SlowLoris

Maintenant, nous pouvons installer Slowloris en téléchargeant son paquet depuis un répertoire GitHub, une plateforme collaborative permettant d'héberger, de gérer et de partager du code source.

```
#git clone https://github.com/gkbrk/slowloris.git
```

Notez qu'avec la commande précédente, *Slowloris* sera installé dans le répertoire actuel, c'est-à-dire celui où vous avez exécuté la commande.

3.Phase d'attaque

Maintenant que Slowloris est installé, nous pouvons le lancer, mais avant cela, il faut récupérer l'adresse IP de la machine cible, ce que j'ai fait avec *ip a*, tandis qu'un attaquant aurait pu utiliser un scan réseau avec Nmap ou un scan d'URL avec Shodan.

```
perl slowloris.pl -dns 192.168.50.127 -port 80
```

Sur cette ligne de commande, nous avons spécifié l'adresse IP du serveur web ainsi que le port sur lequel il écoute.

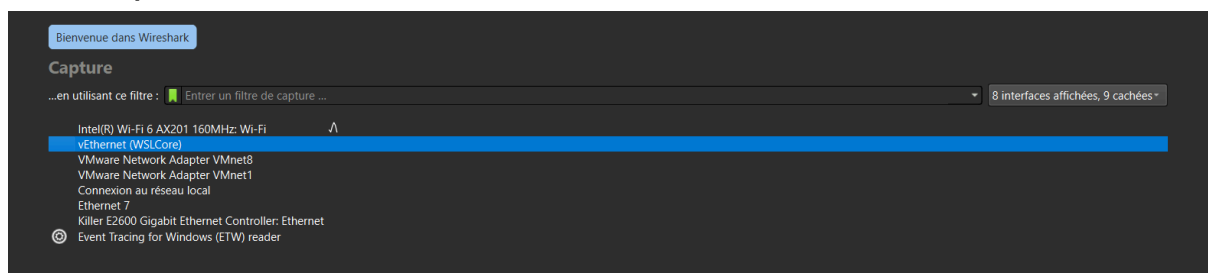
```
[07-03-2025 13:12:59] Attacking 192.168.50.17 with 150 sockets.  
[07-03-2025 13:12:59] Creating sockets...  
[07-03-2025 13:13:02] Sending keep-alive headers...  
[07-03-2025 13:13:02] Socket count: 0  
[07-03-2025 13:13:02] Creating 150 new sockets...
```

4. Analyse du trafic réseau avec Wireshark

Maintenant que l'attaque est lancée, nous pouvons essayer de voir ce qui se trame sur le réseau en utilisant Wireshark pour détecter les requêtes envoyées. Cet outil puissant permet d'analyser le trafic en temps réel et est largement utilisé en cybersécurité pour obtenir un aperçu détaillé des échanges réseau.

4.1 - Utilisation de wireshark

Pour commencer, ouvrez Wireshark et sélectionnez l'interface réseau utilisée pour votre connexion.



Maintenant que vous avez lancé les écoutes, vous pouvez filtrer les recherches par adresse IP en utilisant le filtre suivant dans Wireshark :

```
ip.addr == 192.168.50.197 || ip.addr == 192.168.50.127
```

Une fois cela fait, vous devriez voir apparaître les lignes suivantes :

1	3.792339381	192.168.50.197	192.168.50.127	TCP	77	34502 → 80 [PSH, ACK] Seq=1 Ack=1 Win=501 Len=11
2	3.792459330	192.168.50.127	192.168.50.197	TCP	77	34546 → 80 [PSH, ACK] Seq=1 Ack=1 Win=501 Len=11
3	3.792571958	192.168.50.197	192.168.50.127	TCP	60	80 → 34482 [RST] Seq=1 Win=0 Len=0
4	3.792897885	192.168.50.127	192.168.50.197	TCP	77	34594 → 80 [PSH, ACK] Seq=1 Ack=1 Win=501 Len=11
5	3.793042153	192.168.50.197	192.168.50.127	TCP	77	34606 → 80 [PSH, ACK] Seq=1 Ack=1 Win=501 Len=11
6	3.793704569	192.168.50.197	192.168.50.127	TCP	60	80 → 34604 [RST] Seq=1 Win=0 Len=0

Ces lignes capturées dans Wireshark illustrent une communication TCP entre un attaquant (192.168.50.197) et un serveur web cible (192.168.50.127) sur le port 80. Les paquets avec les drapeaux [PSH, ACK] montrent l'envoi de requêtes fragmentées, une caractéristique typique de l'attaque Slowloris, qui maintient un grand nombre de connexions ouvertes pour saturer les ressources du serveur. En réponse, on observe des paquets [RST], indiquant que le serveur tente de réinitialiser certaines connexions, signe qu'il est potentiellement en train de subir un déni de service en raison d'un trop grand nombre de connexions incomplètes.

5. Installation de ModSecurity

ModSecurity, c'est un pare-feu applicatif web (WAF) qui aide à protéger les serveurs contre les attaques. Dans ce TP, il est particulièrement utile pour contrer *Slowloris*, en filtrant les connexions suspectes et en empêchant le serveur de se retrouver saturé par des requêtes malveillantes. Ici, j'utiliserai le même serveur pour l'installer et tester son efficacité.

5.1 - Installation de ModSecurity

ModSecurity ne fait pas directement partie du paquet Apache2, mais c'est un module additionnel qui doit être installé séparément avec la commande suivante :

```
sudo apt-get install libapache2-mod-security2
```

Ensuite on l'active avec :

```
sudo a2enmod security2
```

Redémarrez Apache :

```
sudo service apache2 restart
```

Vérifier que la configuration a bien été pris en compte :

```
sudo apache2ctl-M | grep security2
```

Télécharger les règles de sécurité :

```
sudo wget  
https://github.com/SpiderLabs/owasp-modsecurity-crs/archive/v  
3.5/master.zip sudo unzip master.zip sudo mv  
owasp-modsecurity-crs-3.5-master /etc/modsecurity.d/owasp-crs  
sudo rm master.zip
```

Ces commandes téléchargent, extraient et déplacent les règles OWASP pour ModSecurity dans le répertoire de configuration, puis suppriment l'archive pour libérer de l'espace.

Copier le fichier dans ModSecurity :

```
sudo cp /etc/modsecurity/modsecurity.conf{-recommended,}
```

Ouvrez ModSecurity :

```
sudo nano /etc/modsecurity.d/modsecurity.conf
```

Et ajouté ces lignes :

```
IncludeOptional modsecurity.d/owasp-crs/crs-setup.conf
```

Pour finir redémarrer Apache :

```
sudo service apache2 restart
```

Conclusion

Ce TP nous a permis de comprendre le fonctionnement de l'attaque Slowloris et son impact sur un serveur web en exploitant les connexions persistantes. À travers des tests pratiques, nous avons observé comment cette attaque peut saturer les ressources du serveur et analyser son trafic avec Wireshark. Enfin, nous avons mis en place ModSecurity, un pare-feu applicatif web, afin de renforcer la sécurité et limiter l'effet de cette menace. Cette expérience met en évidence l'importance des mesures de protection et de la surveillance réseau pour garantir la disponibilité des services.